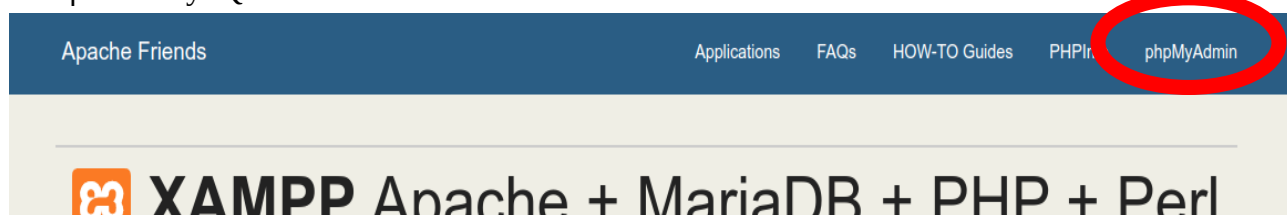


Walkthrough: Διάλεξη 08 - Κλήση απομακρυσμένων λειτουργιών (DB)

Στόχος μας είναι η δημιουργία εφαρμογής η οποία θα συνδέεται σε μια βάση δεδομένων μέσω υπηρεσιών ιστού, θα ανακτά δεδομένα και θα τα εμφανίζει.

Εισαγωγή δεδομένων και σχήματος βάσης δεδομένων

Αφού εκκινήσουμε το XAMPP, και συγκεκριμένα τους διακομιστές Apache και MySQL, μεταβαίνουμε στη διεύθυνση <http://localhost/> και επιλέγουμε το σύνδεσμο πάνω δεξιά, phpMyAdmin, ώστε να μεταβούμε στο ομώνυμο διαδικτυακό εργαλείο διαχείρισης βάσεων δεδομένων MySQL.



Με την έναρξη του phpMyAdmin, επιλέγουμε “Βάσεις Δεδομένων” και στην επόμενη σελίδα που θα ανοίξει

επιλέγουμε “Εισαγωγή”. Στη σελίδα που θα ανοίξει, στο πεδίο “Αρχείο για εισαγωγή” επιλέγουμε το αρχείο **cars2.sql.txt**, που έχουμε μεταφέρει στον υπολογιστή μας από



Αρχείο για εισαγωγή:

Το αρχείο μπορεί να συμπιεστεί (gzip, bzip2, zip) ή να αποσυμπιεστεί.
Το όνομα του συμπιεσμένου αρχείου πρέπει να λήγει σε .[format].[compression]

Περιηγηθείτε στον υπολογιστή σας: (Μέγιστο μέγεθος: 40MB)

Επιλογή αρχείου Δεν επιλέχθηκε κανένα αρχείο.

Μπορείτε να σύρετε και να αφήσετε ένα αρχείο σε οποιαδήποτε σελίδα.

το eClass (Αρχικός κατάλογος > Διάλεξη 08 - Κλήση απομακρυσμένων λειτουργιών (DB) > Δημιουργία Βάσης Δεδομένων Cars (Tuesday)). Έπειτα, επιλέγουμε το πλήκτρο “Εισαγωγή” στο κάτω μέρος της ιστοσελίδας. Το αποτέλεσμα είναι η δημιουργία της βάσης δεδομένων cars, που αποτελείται από δύο πίνακες: τον πίνακα history, όπου θα καταγράφεται το ιστορικό ενεργειών στην εφαρμογή, και τον πίνακα models, ο οποίος περιέχει πληροφορίες για τα οχήματα που θα εμφανίζονται στην εφαρμογή.

Δημιουργία υπηρεσιών

Επόμενο βήμα μας είναι η δημιουργία υπηρεσιών ιστού, οι οποίες θα επικοινωνούν μέσω αιτημάτων HTTP με την εφαρμογή, μεταφέροντας δεδομένα σε μορφή JSON. Έτσι η εφαρμογή θα μπορεί να διαβάζει από και να γράφει στην απομακρυσμένη βάση δεδομένων. Για τη δημιουργία αυτών των υπηρεσιών θα χρησιμοποιήσουμε σενάρια PHP. Κατεβάστε από το eClass το αρχείο “carsDBServices.rar” και αποσυμπιέστε το στη ρίζα εγγράφου του τοπικού διακομιστή ιστού (C:\xampp\htdocs). Θα διαπιστώσετε ότι στο φάκελο carsDBServices που δημιουργήθηκε υπάρχουν δύο αρχεία, ένα για κάθε υπηρεσία. Πρόκειται για τα παρακάτω:

Αρχείο logHistory.php που χρησιμοποιείται για την υπηρεσία καταγραφής ιστορικού:

```
<?php
$data = array();
$brand = $_GET["brand"];
$model = $_GET["model"];
$timestamp = $_GET["timestamp"];

$host="localhost";
$username="root";
```

```

$pass="";
$dbname="cars";

$dbh = mysqli_connect($host,$uname,$pass) or die("cannot connect");
mysqli_select_db($dbh, $dbname);

$sql = "INSERT into history values('' . $brand . "','" . $model . "','" .
$timestamp . "')";
echo $sql;
mysqli_query($dbh, $sql);
mysqli_close($dbh);
?>

```

Αρχείο populateDropDown.php, που χρησιμοποιείται για να παρέχει δεδομένα στα οπτικά στοιχεία της εφαρμογής:

```

<?php
    $data = array();

    $host="localhost";
    $uname="root";
    $pass="";
    $dbname="cars";

    $dbh = mysqli_connect($host,$uname,$pass) or die("cannot connect");
    mysqli_select_db($dbh, $dbname);

    $sql = "SELECT brand, GROUP_CONCAT(model) AS grouped_models FROM models GROUP BY
brand";
    $result = mysqli_query($dbh, $sql);
    while ($row = mysqli_fetch_array($result)) {
        $data[$row['brand']] = $row['grouped_models'];
    }

    header("Content-Type: application/json");
    echo json_encode($data);
    mysqli_close($dbh);
?>

```

ΠΡΟΣΟΧΗ! Πιθανόν να χρειαστεί να αλλάξετε τα στοιχεία χρήστη (**\$uname**, **\$pass**) για να λειτουργήσουν οι υπηρεσίες.

Για να ελέγξετε ότι λειτουργούν οι υπηρεσίες, επισκεφθείτε είτε από περιηγητή ιστού είτε χρησιμοποιώντας τον Postman το <http://localhost/carsDBServices/populateDropDown.php>

Αν όλα λειτουργούν σωστά, θα πρέπει να εμφανίζεται το παρακάτω:

```

{"Nissan":"Sunny", "Toyota":"Auris,Aygo,Corolla,Yaris", "VW":"Golf,Polo"}

```

Δημιουργία frontend

Για τη δημιουργία του frontend θα βασιστούμε σε προηγούμενη υλοποίηση, την οποία βρίσκουμε στην εξής τοποθεσία:

Αρχικός κατάλογος > Διάλεξη 04/05 – Εισαγωγή στο Αντικειμενοστρεφές Mobile Development > Ενδεικτική Λύση

Προετοιμασία

Στη νέα υλοποίηση θα προσπελάσουμε το διαδίκτυο και θα χρησιμοποιήσουμε μια εξωτερική βιβλιοθήκη για να πραγματοποιήσουμε τις κλήσεις HTTP. Για το λόγο αυτό θα πρέπει να προχωρήσουμε σε αλλαγές στα παρακάτω αρχεία:

app > manifests > AndroidManifest.xml

Προσθέστε τα:

```
<uses-permission android:name="android.permission.INTERNET" />
```

και

```
android:usesCleartextTraffic="true"
```

όπως φαίνεται παρακάτω:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission android:name="android.permission.INTERNET" />

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:supportsRtl="true"
        android:theme="@style/Theme.CarPicker"
        android:usesCleartextTraffic="true"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

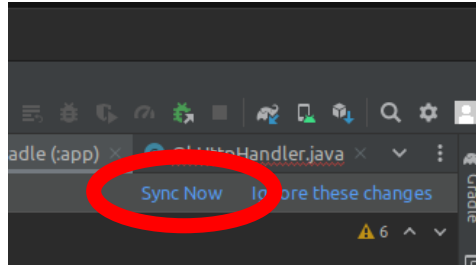
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

Gradle Scripts > build.gradle (module :app)

Κάντε την παρακάτω προσθήκη στα **dependencies**:

```
implementation 'com.squareup.okhttp3:okhttp:4.9.0'
```

Μην ξεχάσετε να επιλέξετε **Sync Now**.



Κώδικας Java

Δημιουργήστε την κλάση OkHttpHandler με τις παρακάτω μεθόδους:

```
public OkHttpHandler() {  
    StrictMode.ThreadPolicy policy = new  
    StrictMode.ThreadPolicy.Builder().permitAll().build();  
    StrictMode.setThreadPolicy(policy);  
}  
  
List<Brand> getCarData(String url) throws Exception {  
    List<Brand> brandsList = new ArrayList<>();  
    OkHttpClient client = new OkHttpClient().newBuilder().build();  
    Request request = new Request.Builder().url(url).build();  
    Response response = client.newCall(request).execute();  
    String data = response.body().string();  
  
    try {  
        JSONObject jsonObject = new JSONObject(data);  
        Iterator<String> keys = jsonObject.keys();  
        while (keys.hasNext()) {  
            String brandName = keys.next();  
            String models = jsonObject.getString(brandName);  
            brandsList.add(new Brand(brandName, models));  
        }  
    } catch (JSONException e) {  
        e.printStackTrace();  
    }  
  
    return brandsList;  
}  
  
public void logHistory(String url) throws Exception {  
    OkHttpClient client = new OkHttpClient().newBuilder().build();  
    Request request = new Request.Builder().url(url).build();  
    Response response = client.newCall(request).execute();  
}
```

Στη συνέχεια πρέπει να εισάγετε την κλάση Request (okhttp3). Δείτε πώς παρακάτω:

```
List<Brand> brandsList = new ArrayList<>();
OkHttpClient client = new OkHttpClient().newBuilder().build();
Request request = new Request.Builder().url(url).build();
Response response = client.newCall(request).execute();
String responseString = response.body().string();
```

Cannot resolve symbol 'Request'

Import class Alt+Shift+Enter More actions... Alt+Enter

```
List<Brand> brandsList = new ArrayList<>();
OkHttpClient client = new OkHttpClient().newBuilder().build();
Request request = new Request.Builder().url(url).build();
```

Class to Import

Request (okhttp3)	Gradle: com.squareup.okhttp3:okhttp:4.9.0 (okhttp-4.9.0.jar)	>
Request in DownloadManager (android.app)	< Android API 33, extension level 3 Platform > (android.jar)	>
Request in VoiceInteractor (android.app)	< Android API 33, extension level 3 Platform > (android.jar)	>
Request in VoiceInteractionSession (android.service.voice)	< Android API 33, extension level 3 Platform > (android.jar)	>

Brand.java

Στην κλάση Brand αντικαταστήστε τον υπάρχοντα κατασκευαστή με τον παρακάτω:

```
public Brand(String brandName, String models) {
    this.brandName = brandName;
    modelsList = Arrays.asList(models.split(", "));
}
```

BrandsList.java

Στην κλάση BrandsList αντικαταστήστε τον υπάρχοντα κατασκευαστή με τον παρακάτω:

```
public BrandsList(String ip) {
    brandsList = new ArrayList<>();
    String url= "http://" + ip + "/carsDBServices/populateDropDown.php";

    try {
        OkHttpHandler okHttpHandler = new OkHttpHandler();
        brandsList = okHttpHandler.getCarData(url);
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

και σβήστε τη λέξη-κλειδί **final** από τη δήλωση της **brandsList**.

MainActivity.java

Προσθέστε ως attribute το παρακάτω:

```
private final String serverNameOrIpAddress = "10.0.2.2";
```

Σημείωση. Η διεύθυνση IPv4 του εικονικού προσαρμογέα του υπολογιστή σας που είναι στο ίδιο υποδίκτυο με τον emulator είναι 10.0.2.2.

Διαγράψτε τις παρακάτω εντολές:

```
brandsList = new BrandsList();

brandsList.addBrand(new Brand("Tomota"));
brandsList.addBrand(new Brand("VR"));
brandsList.addBrand(new Brand("Nistan"));

brandsList.addModel("Tomota", "Kuris");
brandsList.addModel("Tomota", "Yanis");
brandsList.addModel("Tomota", "Corona");
brandsList.addModel("VR", "Wolf");
brandsList.addModel("VR", "Yolo");
```

και αντικαταστήστε τες με αυτήν:

```
brandsList = new BrandsList(serverNameOrIpAddress);
```

Σαν άσκηση βρείτε τις μεθόδους που δεν χρειάζονται πλέον και διαγράψτε τες.

Θυμάστε ότι το Spinner γεμίζει με δεδομένα (μάρκες αυτοκινήτων) από το string-array brands; Στόχος μας είναι όλα τα δεδομένα να ανακτώνται μέσω κλήσεων σε υπηρεσίες ιστού. Γι' αυτό:

- i) στο strings.xml (res > values > strings.xml) διαγράψτε το string-array brands,
- ii) στον κώδικα του layout (res > layout > activity_mail.xml > Code), στη δήλωση του Spinner διαγράψτε την παρακάτω γραμμή:

```
android:entries="@array/brands"
```

- iii) στην κλάση BrandsList δημιουργήστε τη μέθοδο getAllBrandNames:

```
public List<String> getAllBrandNames() {

    List<String> brands = new ArrayList<>();
    for (Brand brand: brandsList)
        brands.add(brand.getBrandName());
    return brands;
}
```

- iv) στο τέλος της μεθόδου onCreate της MainActivity προσθέστε έναν ArrayAdapter που θα γεμίζει με δεδομένα το Spinner:

```
ArrayAdapter<String> arrayAdapter = new ArrayAdapter<>(this,
    android.R.layout.simple_spinner_dropdown_item,
    brandsList.getAllBrandNames());
brands_spinner.setAdapter(arrayAdapter);
```

Αλλάξτε τον παλιό τρόπο κλήσης της μεθόδου showModels (μέσω δήλωσης στην ιδιότητα onClick του κουμπιού στο layout) υλοποιώντας έναν Listener. Κάντε τα παρακάτω:

i) στον κώδικα του layout (res > layout > activity_mail.xml > Code), στη δήλωση του Button διαγράψτε την παρακάτω γραμμή:

```
android:onClick="showModels"
```

ii) στη MainActivity προσθέστε ως attribute το παρακάτω:

```
private Button show_models_button;
```

iii) και στη μέθοδο onCreate της MainActivity αρχικοποιήστε το:

```
show_models_button = findViewById(R.id.show_models_button);
```

iv) στο τέλος της μεθόδου onCreate προσθέστε έναν Listener που θα καλεί τη μέθοδο showModels όταν ο χρήστης πατήσει το κουμπί:

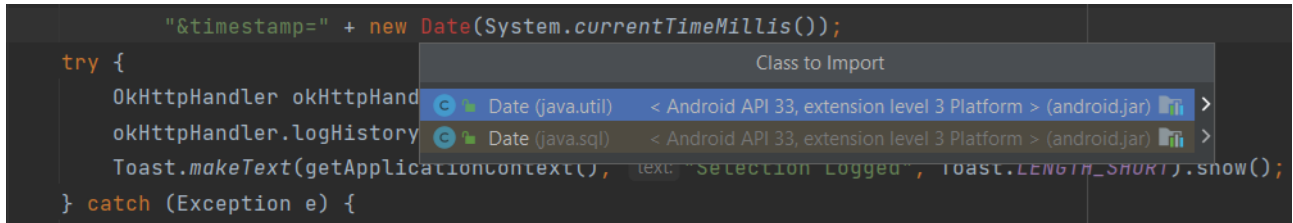
```
show_models_button.setOnClickListener(new View.OnClickListener() {  
    @Override  
    public void onClick(View view) {  
        showModels(view);  
    }  
});
```

Στη συνέχεια, αντικαταστήστε την υπάρχουσα μέθοδο showModels() με την παρακάτω:

```
public void showModels(View view) {  
  
    String brandName = brands_spinner.getSelectedItem().toString();  
    List<String> modelsList = brandsList.getModels(brandName);  
    models_radio_group.removeAllViews();  
  
    for (int i = 0; i < modelsList.size(); i++) {  
        RadioButton radioButton = new RadioButton(this);  
        radioButton.setText(modelsList.get(i));  
        radioButton.setId(i + 100);  
        models_radio_group.addView(radioButton);  
    }  
  
    models_radio_group.setOnCheckedChangeListener(new RadioGroup.OnCheckedChangeListener() {  
        @Override  
        public void onCheckedChanged(RadioGroup radioGroup, int checkedId) {  
            RadioButton radioButton = findViewById(checkedId);  
            String url = "http://" + serverNameOrIpAddress +  
"/carsDBServices/logHistory.php?brand=" + brandName +  
"&model=" + radioButton.getText().toString() +  
"&timestamp=" + new Date(System.currentTimeMillis());  
            try {  
                OkHttpHandler okHttpHandler = new OkHttpHandler();  
                okHttpHandler.logHistory(url);  
                Toast.makeText(getApplicationContext(), "Selection Logged",  
Toast.LENGTH_SHORT).show();  
            } catch (Exception e) {  
                e.printStackTrace();  
            }  
        }  
    });  
}
```

Παρατηρήστε πώς χτίζεται η URL που θα χρησιμοποιηθεί στη μέθοδο `okHttpHandler.logHistory`. Η `logHistory` (σελ. 4) στέλνει τα δεδομένα στο διακομιστή ιστού με την προεπιλεγμένη μέθοδο HTTP GET.

Σημείωση. Θα χρειαστεί να εισάγετε την κλάση `Date`. Δείτε πώς στην παρακάτω εικόνα:



```
"&timestamp=" + new Date(System.currentTimeMillis());  
try {  
    OkHttpHandler okHttpHand  
    okHttpHandler.logHistory  
    Toast.makeText(getApplicationContext(), text: "Selection Logged", Toast.LENGTH_SHORT).show();  
} catch (Exception e) {
```

The screenshot shows an IDE with a Java code snippet. A tooltip titled "Class to Import" is displayed, showing two options: `Date (java.util)` and `Date (java.sql)`. Both options are from the Android API 33, extension level 3 Platform, and are part of the `android.jar`. The `Date (java.util)` option is selected.